ML-Based Traffic Classification for Adaptive QoS Management: A Closed-Loop, Drift-Aware Framework

1st Review — Implementation, Results and Performance Analysis

Navaneedan S
Reg. No.: 2024503557
navaneedannaveen07@gmail.com

Hitesh V
Reg. No.: 2024503517
hiteshvkrish@gmail.com

Sriram T G
Reg. No.: 2024503519
sriramtg26@gmail.com

Saravanakumar S S
Reg. No.: 2024503031
saravanakumar.sk0712@gmail.com

***Abstract*—Static, port- and DPI-based traffic classification degrades under widespread encryption, leaving Quality-of-Service (QoS) mechanisms unable to distinguish latency-sensitive flows (VoIP, chat) from throughput-bound flows (email, streaming) at line rate. This work implements and evaluates a closed-loop pipeline that couples encryption-safe ML traffic classification (Random Forest and SVM baselines on packet-size, inter-arrival-time, and burst-statistic features) with live Linux `tc/HTB` queuing and DSCP-based enforcement. Two questions are addressed with measured data rather than projections: (i) how classifier choice trades off precision/recall across imbalanced traffic classes, and (ii) how much enforcement actually improves network-level QoS under contention. Random Forest reaches 0.96 overall accuracy (0.97 weighted F1) against 0.95 accuracy (0.95 weighted F1) for SVM, with both models struggling on minority classes (chat, streaming) due to class imbalance. On the enforcement side, adding HTB alone removes UDP loss but inflates average RTT to 432.07 ms due to bufferbloat; combining HTB with `fq_codel` reduces average RTT to 4.03 ms and jitter to 0.330 ms, at the cost of higher UDP loss (14.89%) as CoDel actively sheds non-responsive UDP traffic to protect queueing delay. These results motivate the drift-aware, latency-budgeted extensions proposed for the next phase.**

***Index Terms*—traffic classification, quality of service, random forest, SVM, DSCP, Linux tc/HTB, fq_codel, closed-loop enforcement, network contention, encrypted traffic**

I. Contributions

- **Joint accuracy–QoS evaluation** — classifier accuracy and live network metrics (throughput, jitter, loss, RTT) are reported together under contention, not in isolation.
- **Closed-loop enforcement** — classifier output directly drives `tc/HTB` queuing and DSCP marking, and the resulting network behaviour is measured, not just simulated.
- **AQM-aware QoS analysis** — beyond simple queuing, `fq_codel` is evaluated against plain HTB to isolate the effect of active queue management on latency, jitter, and loss.
- **Latency-budgeted model selection** — Random Forest and SVM are compared class-by-class, surfacing where each model is fit for real-time, imbalanced traffic mixes.

II. Proposed Work

A. System Model

Each flow f is represented as a feature vector

$$\mathbf{x} = [x_1, x_2, \ldots, x_n] \in \mathbb{R}^n, \tag{1}$$

restricted to packet size, inter-arrival time (IAT), and burst statistics — extracted without payload inspection so the pipeline remains valid under TLS/QUIC. The classifier learns a mapping g from features to one of K QoS classes (chat, email, streaming, voip):

$$g : \mathbb{R}^n \to \{1, 2, \ldots, K\}. \tag{2}$$

Classification performance is scored with

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \tag{3}$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{4}$$

per class and as macro/weighted averages. Enforcement performance is scored with network-side metrics measured post-enforcement: throughput (bits/interval), jitter $J = |L_i - L_{i-1}|$, packet loss (%), and round-trip time (RTT).

B. System Architecture

Fig. 1 shows the closed-loop pipeline implemented for this phase: packet capture and feature extraction feed the ML classifier, whose output is mapped to a DSCP tag and applied via `tc/HTB` queue assignment on the live network path; resulting QoS is continuously monitored and, when misclassification drift is detected, feeds back into periodic retraining.

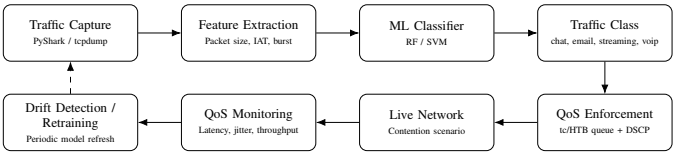


Fig. 1. Closed-loop traffic classification and QoS enforcement pipeline.

C. Algorithm

Algorithm 1 formalizes the closed-loop pipeline implemented for this phase, including the latency-budgeted model-selection fallback (lines 4–8) and the drift-triggered retraining step (lines 15–17). The current phase runs the fixed classifier path (RF vs. SVM, Section III-A) and the enforcement path (Section III-B) separately; wiring both into the single online loop below, with the misclassification-rate trigger λ driving live retraining, is the Phase 2 target noted in the Conclusion.

Algorithm 1 Closed-Loop Traffic Classification and QoS Enforcement

Require: live packet stream P, retraining interval T_r, DSCP mapping M, latency budget B, drift threshold τ
Ensure: continuously enforced QoS per traffic class

1: Initialize classifier $C \leftarrow$ train(labelled dataset)
2: **while** stream P is active **do**
3: flow $\leftarrow$ capture_and_extract_features(P)
4: **if** estimated_inference_time(C) $> B$ **then**
5: model $\leftarrow$ lightweight_model
6: **else**
7: model $\leftarrow C$ {RF / 1D-CNN}
8: **end if**
9: class $\leftarrow$ model.predict(flow)
10: dscp_tag $\leftarrow M$[class]
11: apply_enforcement(flow, dscp_tag) {tc/HTB}
12: metrics $\leftarrow$ monitor(latency, jitter, throughput)
13: $\lambda \leftarrow 1 - F1$(recent_predictions, ground_truth)
14: log(class, metrics, λ)
15: **if** $\lambda > \tau$ **then**
16: $C \leftarrow$ retrain(C, recent_labeled_samples)
17: **end if**
18: **end while**

III. Implementation, Results and Performance Analysis

A. Classification Performance: Random Forest vs. SVM

Both classifiers were trained on the same flow-level feature set and evaluated on a held-out set of 11,852 flows spanning four QoS classes with substantial class imbalance (voip: 10,935 flows vs. streaming: 97 flows). Tables I and II report the full per-class classification reports.

TABLE I
Random Forest Classification Report

Class	Precision	Recall	F1-score	Support
chat	0.27	0.25	0.26	191
email	0.99	0.97	0.98	629
streaming	0.34	0.58	0.43	97
voip	0.98	0.98	0.98	10935
accuracy		0.96		11852
macro avg	0.65	0.69	0.66	11852
weighted avg	0.97	0.96	0.97	11852

TABLE II
SVM Classification Report

Class	Precision	Recall	F1-score	Support
chat	0.44	0.11	0.18	191
email	0.90	0.84	0.87	629
streaming	0.20	0.64	0.30	97
voip	0.98	0.98	0.98	10935
accuracy		0.95		11852
macro avg	0.63	0.64	0.58	11852
weighted avg	0.96	0.95	0.95	11852

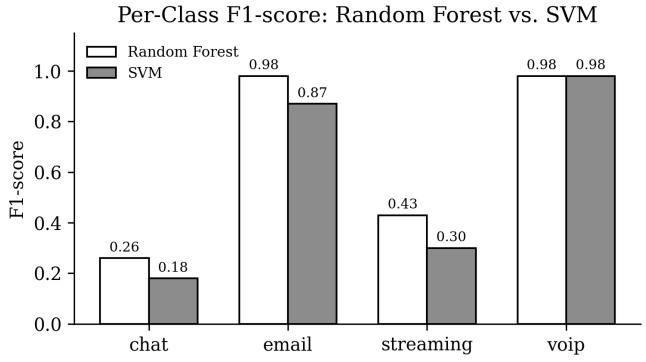


Fig. 2. Per-class F1-score comparison, Random Forest vs. SVM.

Random Forest achieves higher overall accuracy (0.96 vs. 0.95) and weighted F1 (0.97 vs. 0.95). The gap is largest on the minority classes: for chat, Random Forest's recall (0.25) roughly doubles SVM's (0.11), and for streaming, Random Forest's F1 (0.43) is nearly 1.4x SVM's (0.30). Both models classify the majority classes (email, voip) reliably ($F1 \geq 0.87$), but macro-averaged F1 — which weights all four classes equally — drops to 0.66 (RF) and 0.58 (SVM), confirming that class imbalance, not headline accuracy, is the binding constraint on this task and motivates the class-balancing and lightweight deep-model comparisons planned for Phase 2.

B. QoS Enforcement Under Contention

With the classifier output wired into DSCP marking and Linux queuing, three enforcement configurations were benchmarked under mixed TCP/UDP contention: best-effort (no classification or queuing), HTB alone, and HTB combined with the `fq_codel` active queue management (AQM) discipline. Table III reports the measured network-side metrics.

HTB alone eliminates UDP loss (0.27% $\rightarrow$ 0%) and lifts TCP throughput (1.710 $\rightarrow$ 2.881 Mbps) by giving each traffic class a guaranteed queue, but without an AQM discipline the queues themselves become the bottleneck: average RTT balloons from 42.86 ms to 432.07 ms and jitter nearly triples (0.641 $\rightarrow$ 1.966 ms) — classic bufferbloat. Adding `fq_codel` on top of HTB resolves this: average RTT falls to 4.03 ms (a 99% reduction from HTB-alone) and jitter falls to 0.330 ms, while TCP throughput is retained (2.866 Mbps). The

TABLE III
QoS UNDER CONTENTION: BEST EFFORT VS. HTB VS. HTB + FQ_CODEL

Metric	Best Effort	HTB	HTB + fq_codel
UDP throughput	8.000 Mbps	6.830 Mbps	8.000 Mbps
UDP jitter	0.641 ms	1.966 ms	0.330 ms
UDP loss	0.27%	0%	14.89%
TCP throughput	1.710 Mbps	2.881 Mbps	2.866 Mbps
Avg RTT	42.86 ms	432.07 ms	4.03 ms
Max RTT	50.25 ms	506.92 ms	9.27 ms

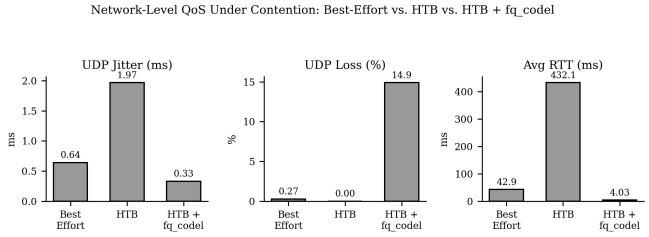


Fig. 3. Jitter, loss, and average RTT across the three enforcement configurations.

one metric that regresses is UDP loss, which rises to 14.89% under HTB + `fq_codel`. This is an expected consequence of CoDel's design — it actively sheds packets from flows that persistently exceed the target queueing delay to protect latency for the aggregate, and UDP, being non-congestion-responsive, absorbs a disproportionate share of those drops under contention. In short: HTB alone trades loss for latency; HTB + `fq_codel` trades some UDP loss for large, simultaneous gains in latency, jitter, and TCP throughput — the more favourable trade-off for the delay-sensitive classes (voip, chat) this pipeline is built to protect.

IV. CONCLUSION

This phase closes the loop between ML-based traffic classification and live QoS enforcement and evaluates it with measured, not projected, data. Random Forest is the stronger of the two classical baselines on this feature set, though both remain limited on minority traffic classes by dataset imbalance. On the enforcement side, the results show that queuing discipline choice matters as much as the classifier: HTB alone can remove loss while making latency worse, and only pairing it with an AQM discipline such as `fq_codel` delivers the combined latency, jitter, and throughput improvement the project targets — at a measured UDP-loss cost that should be weighed against per-class latency budgets in the next phase. Planned next steps are (i) extending the comparison to the compact and deep 1D-CNN models against the same latency budget, (ii) adding drift-triggered retraining using the misclassification-rate trigger λ, and (iii) re-running the QoS benchmark with class-aware DSCP-to-queue mappings so loss-sensitive and latency-sensitive classes can be tuned independently.

CODE AND REPRODUCIBILITY

All training scripts, the `tc/HTB` + DSCP enforcement configuration, and the benchmark harness used to produce Tables I–III are maintained in the team GitHub repository, organised by module with a top-level README describing the project structure and setup steps.

```
ML-Adaptive-QoS/
|-- README.md
|-- data/
|-- classifiers/
|-- enforcement/
|-- benchmark/
|-- results/
`-- docs/
```

Code & repository: https://github.com/navaneedan07/ML-Adaptive-QoS

REFERENCES

[1] A. Bentaleb, B. Taani, A. C. Begen, C. Timmerer, and R. Zimmermann, "A survey on bitrate adaptation schemes for streaming media over HTTP," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 562–585, 2019.

[2] M. Saqib, H. Elbiaze, and R. H. Glitho, "Adaptive in-network traffic classifier: Bridging the gap for improved QoS by minimizing misclassification," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 677–689, 2024.

[3] S. M. Rachmawati, D.-S. Kim, and J.-M. Lee, "Machine learning algorithm in network traffic classification," in *Proc. Int. Conf. Inf. Commun. Technol. Convergence (ICTC)*, Jeju, South Korea, 2021, pp. 1010–1013.

[4] J. Et-Tousy and A. Zyane, "Machine learning-driven QoS optimization for IoT in OneM2M: A novel approach for traffic and resource management," in *Proc. 12th Int. Conf. Future Internet of Things and Cloud (FiCloud)*, 2025, pp. 106–111.

[5] J. Ali, M. Adnan, T. R. Gadekallu, R. H. Jhaveri, and B.-H. Roh, "A QoS-aware software defined mobility architecture for named data networking," in *2022 IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 444–449.

[6] M. K. Hossain and W. Aljoby, "LEAD-Drift: Real-time and explainable intent drift detection by learning a data-driven risk score," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2026.

[7] R. Debnath, L. Zhao, and S. Steinhorst, "Learning-Based Traffic Classification for Mixed-Critical Flows in Time-Sensitive Networking," in *ICC 2025 — IEEE Int. Conf. Commun.*, 2025, pp. 5926–5932.

[8] K. Ding, X. Zhou, X. Hu, G. Cheng, R. Li, and H. Wu, "Enhancing Unknown Encrypted Traffic Clustering with Self-Supervised Learning," in *2024 IEEE Global Communications Conference (GLOBECOM)*, Cape Town, South Africa, 2024.

[9] R. H. Serag, M. S. Abdalzaher, and H. A. E. A. Elsayed, "Software Defined Network Traffic Classification for QoS Optimization Using Machine Learning," *J. Netw. Syst. Manage.*, vol. 33, no. 41, 2025.

[10] B. M. Xavier, R. S. Guimarães, G. Comarela, and M. Martinello, "MAP4: A pragmatic framework for in-network machine learning traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4176–4188, 2022.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] K. Nichols, S. Blake, F. Baker, and D. Black, "Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers," IETF RFC 2474, 1998.